

CAPSTONE PROJECT – 2

Semiconductor Manufacturing Process Yield Prediction Using Machine Learning

Group No:3

Team Members

Name: Yash Raj

Mobile No.: 7979045949

Name: Bharathkumar B

Mobile No.: 6374427735

Name: J. Shashi Kumar

Mobile No.: 8317652950

Name: Dwarampudi Vaibhav Reddy

Mobile No.: 8297336688

Domain: Semiconductor Manufacturing Process

Project Type: Machine Learning Classification

Project Completion: 2026

Executive Summary

The semiconductor manufacturing industry is a highly complex and precision-driven domain where product quality depends on multiple controlled process parameters. Modern production lines are equipped with a large number of sensors that continuously collect data at different stages of manufacturing. These sensor readings provide valuable insights into machine performance and process stability.

However, real-world sensor data is often high-dimensional and may include redundant features, missing values, and noisy measurements. Not all sensor signals contribute equally to predicting the final product quality, which makes the analysis challenging and computationally expensive.

In this project, a machine learning-based classification approach is developed to predict the final outcome of the semiconductor manufacturing process. The objective is to classify each production instance as either **Pass (-1)** or **Fail (1)** using historical sensor data.

The project follows a structured data science workflow. It begins with data exploration and understanding, followed by preprocessing steps such as handling missing values, data cleaning, and feature analysis. Since the dataset is imbalanced and high-dimensional, appropriate techniques are applied to improve data quality and model performance.

Feature selection and dimensionality reduction techniques are used to reduce complexity and retain only the most relevant sensor variables. This helps in improving model efficiency and reducing overfitting.

Multiple machine learning algorithms such as Random Forest, Support Vector Machine, and Naive Bayes are trained and evaluated using cross-validation and standard evaluation metrics like accuracy, precision, recall, and F1-score. Hyperparameter tuning is applied to improve model performance.

The final selected model provides reliable predictions of manufacturing outcomes and can assist engineers in early detection of defects, improving process efficiency, reducing production loss, and enhancing overall yield in semiconductor manufacturing systems.

Business Problem

Semiconductor manufacturing is a highly sensitive and precision-based industry where product quality depends on strict control of multiple process parameters such as temperature, pressure, chemical composition, and equipment stability. Even minor variations in these parameters can lead to significant production defects and financial losses.

In modern manufacturing systems, hundreds of sensors are deployed across different stages of production to continuously monitor process behavior. These sensors generate large volumes of high-dimensional data in real time. While this data is valuable, it often contains redundant, irrelevant, and noisy features that make analysis more complex.

The main business challenge is to develop a reliable predictive system that can accurately determine whether a semiconductor production unit will pass or fail the final quality inspection based on historical sensor data.

Traditional manual inspection methods or rule-based systems are not sufficient to handle such complex and large-scale datasets. They are time-consuming, less accurate, and unable to capture hidden patterns in data.

Incorrect or delayed detection of defective products can result in increased production costs, material wastage, reduced yield, and inefficiency in the manufacturing process. Therefore, there is a strong requirement for an automated, data-driven solution that can support early fault detection.

By applying machine learning techniques, it is possible to analyze complex sensor patterns and build a classification model that can accurately distinguish between normal and faulty production outcomes. This helps in improving process efficiency, reducing operational losses, and enhancing overall production quality.

Importing Required Libraries

Before performing data analysis and model development, it is essential to import all the required Python libraries.

The imported libraries provide functionalities for:

- Data loading and manipulation
- Data visualization and Exploratory Data Analysis (EDA)
- Missing value treatment and data preprocessing
- Feature scaling and train-test splitting
- Handling class imbalance using SMOTE
- Building machine learning models
- Hyperparameter tuning using GridSearchCV
- Model evaluation using various performance metrics
- Cross-validation for model validation

- Suppressing unnecessary warnings for a clean notebook output

These libraries form the foundation for implementing the complete machine learning pipeline in this project.

```
In [8]: # basic libraries for data handling
import numpy as np
import pandas as pd

# visualization libraries for graphs and plots
import matplotlib.pyplot as plt
import seaborn as sns

# splitting dataset and model validation
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score

# scaling data for better model performance
from sklearn.preprocessing import StandardScaler

# handling missing values if present in dataset
from sklearn.impute import SimpleImputer

# evaluation metrics to measure model performance
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

# machine learning models used in this project
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

# used for handling imbalanced dataset (important for classification)
from imblearn.over_sampling import SMOTE

# to ignore warnings and keep output clean
import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully")
```

Libraries imported successfully

Loading the Dataset

In this step, the semiconductor manufacturing dataset is loaded into a Pandas DataFrame. After loading, the dataset size is verified, and a sample of the records is displayed to understand the overall structure of the data before performing further analysis.

```
In [16]: # Loading the dataset using local system path
df = pd.read_csv(r"C:\Users\Yash Raj\Desktop\signal-data.csv")

# confirming dataset is Loaded
print("Dataset loaded successfully")

# checking dataset size (rows, columns)
print("Dataset shape:", df.shape)

# viewing sample records to understand structure
df.head(15)
```

Dataset loaded successfully

Dataset shape: (1567, 592)

Out[16]:

	Time	0	1	2	3	4	5	6	7	8	...	581	582	583	584
0	19-07-2008 11:55	3030.93	2564.00	2187.7333	1411.1265	1.3602	100.0	97.6133	0.1242	1.5005	...	NaN	0.5005	0.0118	0.0035
1	19-07-2008 12:32	3095.78	2465.14	2230.4222	1463.6606	0.8294	100.0	102.3433	0.1247	1.4966	...	208.2045	0.5019	0.0223	0.0055
2	19-07-2008 13:17	2932.61	2559.94	2186.4111	1698.0172	1.5102	100.0	95.4878	0.1241	1.4436	...	82.8602	0.4958	0.0157	0.0039
3	19-07-2008 14:43	2988.72	2479.90	2199.0333	909.7926	1.3204	100.0	104.2367	0.1217	1.4882	...	73.8432	0.4990	0.0103	0.0025
4	19-07-2008 15:22	3032.24	2502.87	2233.3667	1326.5200	1.5334	100.0	100.3967	0.1235	1.5031	...	NaN	0.4800	0.4766	0.1045
5	19-07-2008 17:53	2946.25	2432.84	2233.3667	1326.5200	1.5334	100.0	100.3967	0.1235	1.5287	...	44.0077	0.4949	0.0189	0.0044
6	19-07-2008 19:44	3030.27	2430.12	2230.4222	1463.6606	0.8294	100.0	102.3433	0.1247	1.5816	...	NaN	0.5010	0.0143	0.0042
7	19-07-2008 19:45	3058.88	2690.15	2248.9000	1004.4692	0.7884	100.0	106.2400	0.1185	1.5153	...	95.0310	0.4984	0.0106	0.0034
8	19-07-	2967.68	2600.47	2248.9000	1004.4692	0.7884	100.0	106.2400	0.1185	1.5358	...	111.6525	0.4993	0.0172	0.0046

	Time	0	1	2	3	4	5	6	7	8	...	581	582	583	584
	2008 20:24														
9	19- 07- 2008 21:35	3016.11	2428.37	2248.9000	1004.4692	0.7884	100.0	106.2400	0.1185	1.5381	...	90.2294	0.4967	0.0152	0.0038
10	19- 07- 2008 21:57	2994.05	2548.21	2195.1222	1046.1468	1.3204	100.0	103.3400	0.1223	1.5144	...	57.8122	0.4925	0.0158	0.0041
11	19- 07- 2008 22:52	2928.84	2479.40	2196.2111	1605.7578	0.9959	100.0	97.9156	0.1257	1.4690	...	75.5077	0.4987	0.0427	0.0092
12	20- 07- 2008 03:35	2920.07	2507.40	2195.1222	1046.1468	1.3204	100.0	103.3400	0.1223	1.5310	...	52.2039	0.4950	0.0153	0.0041
13	21- 07- 2008 08:21	3051.44	2529.27	2184.4333	877.6266	1.4668	100.0	107.8711	0.1240	1.5236	...	NaN	0.5034	0.0151	0.0038
14	21- 07- 2008 11:53	2963.97	2629.48	2224.6222	947.7739	1.2924	100.0	104.8489	0.1197	1.4474	...	142.9080	0.5077	0.0094	0.0026

15 rows × 592 columns

Observation

- The dataset was loaded successfully without any errors.

- It contains **1,567 observations (rows)** and **592 columns**.
- Among these, **591 columns represent sensor measurements**, while the remaining column is the target variable (**Pass/Fail**).
- Displaying the first 15 records provides an initial understanding of the dataset structure, feature values, and data format, which is useful before starting data preprocessing and exploratory data analysis.

EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is performed to understand the structure of the dataset, identify patterns, detect missing values, and gain insights before building machine learning models.

This step helps in:

- Understanding dataset structure
- Checking data quality
- Identifying class imbalance
- Preparing data for preprocessing and modeling

```
In [17]: # checking dataset structure and information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1567 entries, 0 to 1566
Columns: 592 entries, Time to Pass/Fail
dtypes: float64(590), int64(1), object(1)
memory usage: 7.1+ MB
```

Dataset Structure Analysis

From the dataset information, the following observations are made:

- The dataset contains **1567 rows (records)** and **592 columns (features)**.
- There is one target column: **Pass/Fail**.
- Most of the features are of type **float64**, indicating continuous sensor measurements.
- One feature is of type **int64**, and one is of type **object**, which may require further inspection.

- The dataset size is approximately **7.1+ MB**, indicating a moderately large dataset.

Key Insight:

- Since the dataset is high-dimensional, feature selection or dimensionality reduction may be required before model building.
- Data preprocessing will play a crucial role in improving model performance.

```
In [18]: # checking total missing values in dataset
missing_values = df.isnull().sum().sum()
print("Total Missing Values in Dataset:", missing_values)
```

Total Missing Values in Dataset: 41951

Missing Values Analysis

To understand data quality, the dataset was checked for missing values.

Result:

- Total Missing Values in Dataset: **41,951**

Observation:

- A significant number of missing values are present in the dataset.
- These missing values are spread across multiple sensor features.
- This is expected in real-world manufacturing environments due to sensor failure or data recording issues.

Key Insight:

- Missing values must be handled before model building.
- Proper preprocessing techniques such as imputation or feature treatment will be applied in the next stage.

```
In [19]: # statistical summary of dataset features
df.describe().T
```

Out[19]:

	count	mean	std	min	25%	50%	75%	max
0	1561.0	3014.452896	73.621787	2743.2400	2966.260000	3011.4900	3056.6500	3356.3500
1	1560.0	2495.850231	80.407705	2158.7500	2452.247500	2499.4050	2538.8225	2846.4400
2	1553.0	2200.547318	29.513152	2060.6600	2181.044400	2201.0667	2218.0555	2315.2667
3	1553.0	1396.376627	441.691640	0.0000	1081.875800	1285.2144	1591.2235	3715.0417
4	1553.0	4.197013	56.355540	0.6815	1.017700	1.3168	1.5257	1114.5366
...	...	...	...	...	...	...	...	...
586	1566.0	0.021458	0.012358	-0.0169	0.013425	0.0205	0.0276	0.1028
587	1566.0	0.016475	0.008808	0.0032	0.010600	0.0148	0.0203	0.0799
588	1566.0	0.005283	0.002867	0.0010	0.003300	0.0046	0.0064	0.0286
589	1566.0	99.670066	93.891919	0.0000	44.368600	71.9005	114.7497	737.3048
Pass/Fail	1567.0	-0.867262	0.498010	-1.0000	-1.000000	-1.0000	-1.0000	1.0000

591 rows × 8 columns

Statistical Summary Analysis

The statistical summary provides an overview of the numerical features in the dataset.

Observations

- The **count** of observations is not the same for all features. Some columns have fewer than **1567** observations, confirming the presence of missing values.
- The **mean**, **median (50%)**, and **standard deviation** differ considerably across features, indicating that sensor measurements are recorded on different scales.
- Several features have a wide difference between their minimum and maximum values, suggesting the possible presence of outliers.